AI PRE-PLACEMENT TRAINER
& FEEDBACK MODEL

Complete Project Blueprint — Industry-Ready Implementation Guide

Project Name	AI-Based Pre-Placement Trainer and Feedback Model
Version	1.0 — Full System Blueprint
Frontend Stack	React 19 + TypeScript + Vite + TailwindCSS + Zustand
Backend Stack	Django 4.2 + Django REST Framework + SimpleJWT
AI Engine	Google Gemini 2.5 Flash (google-generativeai SDK)
Database	SQLite (dev) -> PostgreSQL (production)
Total Modules	5 — Auth, Learning, Aptitude Quiz, AI Interview, Dashboard
Total Bugs Found	11 Critical + 6 Enhancement Gaps (documented in Section 4)
Prepared By	Vedant Kumbhar, Meeraj Krishna M.R., Loukik Ingale
Guide	Dr. Suraj R. Nalawade
Institution	YSPM's Yashoda Technical Campus, Satara
Date	March 2026

YSPM's Yashoda Technical Campus, Satara | Department of AI & Data Science | Academic Year 2025-26

1. Project Overview & Architecture Summary

This blueprint is based on a complete study of every file in the AI-BASED-TRAINER-FEEDBACK-MODEL-main project ZIP. It documents the exact current state of the codebase, all bugs found by reading the source code, and the precise steps needed to build every module to industry standard.

1.1 What the Project Does

The AI Pre-Placement Trainer is a web application that helps engineering students prepare for campus placement interviews. It has five functional modules:

- Auth Module — Registration and login using Django SimpleJWT + custom User model
- Learning Module — Topic-wise aptitude learning content with rich HTML descriptions and embedded YouTube videos
- Aptitude Quiz Module — MCQ-based quiz per topic, scored with correct/wrong tracking
- AI Interview Module — Resume upload -> resume parsing -> AI-generated personalized questions -> voice/text answers -> AI holistic evaluation -> results + PDF
- Dashboard Module — Performance summary, history, stats charts

1.2 Confirmed Technology Stack

Layer	Technology	Version	File Confirmed In
Frontend Framework	React + TypeScript	19.2.0	package.json
Build Tool	Vite (rolldown-vite)	7.2.5	package.json
Styling	TailwindCSS	3.4.17	tailwind.config.js
State Management	Zustand	5.0.9	package.json
HTTP Client	Axios	1.13.2	package.json
Charts	Recharts	3.6.0	package.json
Icons	Lucide React	0.562.0	package.json
Router	React Router DOM	7.11.0	package.json
Backend Framework	Django	4.2.0	requirements.txt
REST API	Django REST Framework	3.14.0	requirements.txt
Auth Tokens	SimpleJWT	5.3.0	requirements.txt
CORS	django-cors-headers	4.2.0	requirements.txt
AI — Gemini	google-generativeai	0.3.0 -> upgrade to 0.7+	requirements.txt
PDF Parsing	PyPDF2	3.0.1	requirements.txt
Word Parsing	python-docx	0.8.11	requirements.txt
Task Queue	Celery + Redis	5.3.1 / 5.0.0	requirements.txt

2. Complete Project Folder Structure

2.1 Frontend — src/ Structure

File / Path	Status	Notes
frontend/src/App.tsx	✅EXISTS	Routes defined correctly
pages/Dashboard.tsx	✅EXISTS	Dashboard page
pages/Learning.tsx	✅EXISTS	Reads from static aptitudeData.ts
pages/Quiz.tsx	✅EXISTS	Uses static aptitudeData.ts
pages/InterviewSession.tsx	⚠ BUG	Evaluates per-answer, no TTS await, no silence detection
pages/InterviewResults.tsx	❌MISSING	Needs full results dashboard — must create
pages/InterviewPDFReport.tsx	❌MISSING	PDF download page — must create
components/auth/Register.tsx	⚠ BUG	No name regex validation
components/interview/BrowserCheck.tsx	❌MISSING	Must create
components/interview/MicPermission.tsx	❌MISSING	Must create
hooks/useTTS.ts	❌MISSING	Text-to-speech hook — must create
hooks/useSTT.ts	❌MISSING	Speech-to-text hook — must create
hooks/useSilenceDetector.ts	❌MISSING	Must create
hooks/useInterviewSession.ts	❌MISSING	localStorage persistence — must create
services/api.ts	✅EXISTS	axiosInstance with JWT interceptor
services/interviewAPI.ts	⚠ BUG	Uses per-answer endpoint, needs new submit-all
store/authStore.ts	⚠ BUG	Stores only access token, no refresh logic
data/aptitudeData.ts	⚠ STATIC	Static data only, no API calls, only 5 Q per topic
utils/ folder	❌MISSING	pdfGenerator.ts, browserSupport.ts, validators.ts

2.2 Backend — apps/ Structure

File / Path	Status	Notes
backend/manage.py	✅EXISTS	Django entry
backend/requirements.txt	⚠ OUTDATED	google-generativeai version 0.3.0
ai_trainer/settings.py	⚠ NEEDS UPDATE	GEMINI_API_KEY config needed

ai_trainer/urls.py	✅EXISTS	aptitude/learning URLs commented out
apps/accounts/models.py	✅EXISTS	CustomUser extends AbstractUser
apps/accounts/views.py	✅EXISTS	Register, Login, Logout, Profile
apps/interview/models.py	✅EXISTS	Resume, InterviewSession, Question, Answer, Feedback
apps/interview/views.py	□ BUG	Evaluates one answer per API call
apps/interview/services/gemini_service.py	✅EXISTS	generate_question, evaluate_answer, session_feedback
apps/interview/services/whisper_service.py	□ NOTE	Whisper API used — NOT Web Speech API
apps/aptitude/	□ EMPTY	models.py empty, no views, URL commented out
apps/learning/	□ EMPTY	models.py empty, no views, URL commented out
apps/dashboard/	□ EMPTY	models.py empty, no views

3. Complete Database Schema

All tables are defined in Django models. This section documents every table, every column, its type, and its purpose.

3.1 custom_user Table (accounts/models.py)

Column	Type	Nullable	Purpose
id	AutoField PK	No	Auto-incrementing user ID
username	CharField(150)	No	Unique username
email	EmailField	No	Login email
first_name	CharField(150)	Yes	First name
last_name	CharField(150)	Yes	Last name
password	CharField	No	bcrypt hashed password
bio	TextField	Yes	User biography
profile_picture	ImageField	Yes	Upload path: profiles/
phone_number	CharField(15)	Yes	Mobile number
is_active	BooleanField	No	Account active flag
created_at	DateTimeField	No	auto_now_add
updated_at	DateTimeField	No	auto_now

3.2 interview_resume Table

Column	Type	Purpose
id	UUIDField PK	uuid4 auto-generated
user_id	FK -> custom_user	Owner of resume
file	FileField	Upload path: resumes/
filename	CharField(255)	Original filename
raw_text	TextField	Full extracted text from PDF
skills	JSONField (list)	['Python', 'React', 'ML', ...]
experience	JSONField (list)	{title, company, duration, highlights}
education	JSONField (list)	{degree, institution, year}
projects	JSONField (list)	{name, description, technologies}
summary	TextField	AI-generated candidate summary
is_parsed	BooleanField	True when Gemini parsing complete
created_at	DateTimeField	auto_now_add

3.3 interview_session Table

Column	Type	Values / Purpose
id	UUIDField PK	uuid4 auto-generated
user_id	FK -> custom_user	Session owner
resume_id	FK -> resume (nullable)	Linked resume, null for quick interview
interview_type	CharField(20)	'HR' | 'Technical' | 'Behavioral' | 'Mixed'
total_questions	PositiveIntegerField	Default 5, range 3-10
status	CharField(20)	'pending' | 'in_progress' | 'completed' | 'cancelled'
current_question_index	PositiveIntegerField	Tracks progress (1-indexed)
start_time	DateTimeField (nullable)	Set when session starts
end_time	DateTimeField (nullable)	Set when session ends
overall_score	FloatField (nullable)	0-100, set at session end
communication_score	FloatField (nullable)	0-100
technical_score	FloatField (nullable)	0-100
confidence_score	FloatField (nullable)	0-100
created_at	DateTimeField	auto_now_add

3.4 interview_answer Table

Column	Type	Purpose
id	UUIDField PK	uuid4
question_id	OneToOneField -> question	One answer per question
answer_text	TextField	User's spoken/typed response
answer_duration_seconds	PositiveIntegerField (nullable)	Time taken to answer
score	FloatField (0-100)	Per-answer score from Gemini
ai_feedback	TextField	2-3 sentence specific feedback
strengths	JSONField (list)	['strength 1', ...]
improvements	JSONField (list)	['improvement 1', ...]
key_points_covered	JSONField (list)	Expected points the user mentioned
key_points_missed	JSONField (list)	Expected points the user missed
relevance_score	FloatField (nullable)	0-100

clarity_score	FloatField (nullable)	0-100
depth_score	FloatField (nullable)	0-100
created_at	DateTimeField	auto_now_add

3.5 interview_feedback Table (Session-level)

Column	Type	Purpose
id	UUIDField PK	uuid4
session_id	OneToOneField -> session	One feedback per session
overall_summary	TextField	4-5 sentence holistic assessment
overall_rating	CharField(20)	'excellent' | 'good' | 'average' | 'needs_improvement' | 'poor'
strengths	JSONField (list)	Overall strengths across all answers
weaknesses	JSONField (list)	Areas for improvement
suggestions	JSONField (list)	3 actionable improvement suggestions
topic_scores	JSONField (dict)	{'communication':80, 'technical_knowledge':75, ...}
percentile	FloatField (nullable)	Percentile vs other users
recommended_resources	JSONField (list)	Learning resources to study
practice_areas	JSONField (list)	Topics to focus on next
created_at	DateTimeField	auto_now_add

4. Complete Bug Inventory — Found by Code Review

Every bug listed below was confirmed by reading the actual source code in the project ZIP. File names and exact locations are given for every bug.

ID	File	Bug Description	Severity	Fix Summary
BUG-01	Register.tsx	fullName input has no character validation. handleChange sets state directly with no regex check. Numbers and symbols are accepted.	**Critical**	Add regex /^[a-zA-Z\s'-]{2,50}$/ in onChange; block invalid chars
BUG-02	serializers.py	Backend serializer has no server-side name validation. Only email/password checked. SQL injection possible via name field.	**Critical**	Add name field to RegisterSerializer.validate() with regex check
BUG-03	aptitudeData.ts	Quiz questions hardcoded in static TypeScript file (5 per topic). getQuestionsByTopicId() always returns the same 5. No API call, no randomization.	**Critical**	Move questions to backend DB; add ORDER BY RANDOM() + pagination
BUG-04	InterviewSession.tsx	submitAnswer() calls InterviewAPI.submitAnswer() per answer, hitting /submit-answer/. Each call waits 10-30s for Gemini. User sees 'Analyzing...' 5 times.	**Critical**	Redesign: collect all answers, send in single POST to /submit-all/
BUG-05	InterviewSession.tsx	startRecording() uses MediaRecorder API — records audio blob and sends to /transcribe/ (Whisper). Adds latency AND requires server-side Whisper. No fallback.	**Critical**	Replace with Web Speech API (browser-native); use Whisper only as fallback
BUG-06	InterviewSession.tsx	speakQuestion() has no Chrome 60s TTS pause-resume fix. Long questions will pause mid-sentence on Chrome (known browser bug).	**High**	Add setInterval keep-alive: if(speechSynthesis.paused) speechSynthesis.resume()
BUG-07	InterviewSession.tsx	TTS voice selected using find() before getVoices() is populated. On first load, voices[] is empty — utterance plays in default robotic voice.	**High**	Add speechSynthesis.onvoiceschanged listener before selecting voice
BUG-08	InterviewSession.tsx	No silence detection. After user stops speaking, nothing happens until they manually click 'Submit Answer'. Interview flow entirely manual.	**Critical**	Add useSilenceDetector hook: 3s of no new transcript auto-advances

BUG-09	InterviewSession.tsx	No localStorage backup of answers. If user refreshes page mid-interview, all answers and session context are lost.	**High**	Save each answer to localStorage on finalize; restore on mount
BUG-10	views.py (interview)	submit_answer view generates next question INSIDE same request. If question generation fails, answer is still saved but user sees error. Atomic issue.	**Medium**	Decouple answer saving and next question generation into separate steps
BUG-11	interviewAPI.ts	API_BASE_URL hardcoded as 'http://localhost:8000/api/interview'. Will break in production.	**High**	Use import.meta.env.VITE_API_BASE_URL from environment variables
BUG-12	authStore.ts	setAuth() stores only access_token, not refresh_token. Token refresh logic reads refresh from localStorage directly — inconsistency.	**Medium**	Unify: store both tokens in Zustand or both in localStorage, not split
BUG-13	gemini_service.py	google-generativeai version 0.3.0 is outdated. Current stable is 0.7+. API method signatures have changed.	**High**	Update to google-generativeai>=0.7.0 in requirements.txt
BUG-14	apps/aptitude/	Aptitude app exists but has no models.py content, no views.py, no URLs. Aptitude quiz has zero backend support.	**High**	Implement AptitudeTopic and AptitudeQuestion models + seeder + API
BUG-15	apps/learning/	Learning app same as aptitude — empty app, URL commented out. Learning content hardcoded in aptitudeData.ts.	**Medium**	Implement LearningTopic model + admin panel + API
BUG-16	InterviewFeedback.tsx	Score displayed as out of 100 but Gemini evaluation prompt returns scores 0-10. Values displayed 10x too small unless conversion exists.	**Medium**	Standardize: use 0-10 in DB and multiply by 10 only at display layer

5. Complete API Endpoint Specification

All endpoints currently in the project, plus the new /submit-all endpoint that must be added. Base URL: http://localhost:8000/api/

5.1 Auth Endpoints (api/auth/)

Method	URL	Auth	Request Body	Response	Status
POST	/auth/register/	None	username, email, first_name, last_name, password, password_confirm	access, refresh, user{}	201
POST	/auth/login/	None	email, password	access, refresh, user{}	200
POST	/auth/logout/	Bearer JWT	—	message: 'Logged out'	200
GET	/auth/profile/	Bearer JWT	—	User object	200
PUT	/auth/profile/	Bearer JWT	Partial user fields	Updated user	200
POST	/auth/token/refresh/	None	refresh token	new access token	200

5.2 Interview Endpoints (api/interview/)

Method	URL	Auth	Request	Response	Status
GET	interview/resume/	JWT	—	List of user resumes	200
POST	interview/resume/	JWT	multipart file	Resume object with parsed data	201
GET	interview/resume/{id}/	JWT	—	Full resume detail	200
POST	interview/start/	JWT	interview_type, resume_id?, total_questions	session{}, current_question{}	201
GET	interview/question/{session_id}/	JWT	—	question{}, questions_remaining	200
POST	interview/submit-answer/	JWT	session_id, question_id, answer_text, duration_seconds?	answer_feedback{}, next_question{}	200
POST	interview/end/{session_id}/	JWT	—	session{}, overall_feedback{}	200
GET	interview/feedback/{session_id}/	JWT	—	Full session with all QA + feedback	200
GET	interview/history/	JWT	type?, status?, page?	Paginated session list	200

GET	interview/stats/	JWT	—	total, avg_score, best_score, by_type{}	200
DELETE	interview/{session_id}/	JWT	—	204 No Content	204
POST	interview/transcribe/	JWT	multipart audio	success, text	200
POST	interview/submit-all/	JWT	session_id, answers[]	MISSING — must create	—

NEW ENDPOINT: POST /api/interview/submit-all/ — Accepts ALL answers at once. Sends them together to Gemini in ONE prompt. Returns holistic evaluation with: overall_score, placement_readiness, summary, top_strength, top_weakness, recommendations[], scores{}, question_results[]. This replaces 8 sequential calls to /submit-answer/ with a single call at session end. Gemini temperature: 0.2, max_output_tokens: 3000.

5.3 Aptitude Endpoints (to be created)

Method	URL	Purpose
GET	aptitude/topics/	List all aptitude topics with metadata
GET	aptitude/topics/{id}/	Get topic detail including rich HTML content
GET	aptitude/questions/?topic_id=&count=10	Get N random questions for topic (ORDER BY RANDOM())
POST	aptitude/submit/	Submit answers, get score and per-question feedback
GET	aptitude/results/{attempt_id}/	Get detailed result for one quiz attempt
GET	aptitude/history/	User quiz history with scores per topic

5.4 Learning Endpoints (to be created)

Method	URL	Purpose
GET	learning/topics/	List all learning topics
GET	learning/topics/{id}/	Get full topic content (HTML + video_url + references)
GET	learning/progress/	User topic completion status
POST	learning/progress/{topic_id}/mark-read/	Mark topic as read/completed

6. Module Blueprint — Authentication

The auth module is mostly functional. The main issues are missing name validation and split state management between Zustand and localStorage.

6.1 Current State

- PASS: Register endpoint works — validates email uniqueness and password match
- PASS: Login endpoint works — returns access and refresh JWT tokens
- PASS: ProtectedRoute.tsx checks AuthService.isAuthenticated() before routing
- FAIL BUG-01: Register.tsx — fullName input has no validation at all
- FAIL BUG-02: RegisterSerializer — no server-side name format validation
- WARN BUG-12: authStore.ts stores only access token — inconsistent with authService

6.2 Register.tsx — Name Validation Fix

In Register.tsx, find the handleChange function and replace it with this:

```
// Add at the top of the Register component:
const [nameError, setNameError] = useState('');
const NAME_REGEX = /^[a-zA-Z\s'\-]{2,50}$/;

// Replace fullName input onChange handler:
onChange={(e) => {
  const val = e.target.value;
  if (/^[a-zA-Z\s'\-]*$/.test(val) || val === '') {
    setFormData({ ...formData, fullName: val });
    if (val.length > 0 && !NAME_REGEX.test(val)) {
      setNameError('Name must be 2-50 letters only. No numbers or symbols.');
    } else {
      setNameError('');
    }
  }
}}

// In handleSubmit, before API call:
if (!NAME_REGEX.test(formData.fullName.trim())) {
  setError('Please enter a valid name (letters only).');
  return;
}
```

6.3 Backend Serializer — Name Validation Fix

In apps/accounts/serializers.py, add to the RegisterSerializer.validate() method:

```
import re

def validate(self, data):
    # BUG-02 FIX: Validate name format
    name_fields = ['first_name', 'last_name']
    for field in name_fields:
        value = data.get(field, '').strip()
        if value and not re.match(r"^[a-zA-Z\s'\-]{1,50}$", value):
            raise serializers.ValidationError({
                field: f'{field.replace(" ", " ").title()} must contain only letters.'
            })
    if data['password'] != data['password_confirm']:
        raise serializers.ValidationError({'password': 'Passwords do not match'})
```

7. Module Blueprint — Learning & Aptitude Quiz

Both modules currently work from static TypeScript data. The content quality is good but the system cannot be updated without changing code, has no admin panel, and the quiz has no randomization.

7.1 Current State

- PASS: aptitudeData.ts has rich content for multiple topics including full HTML theory, video URLs, question sets
- PASS: Quiz UI (Quiz.tsx, QuestionCard.tsx, QuizProgress.tsx, QuizTimer.tsx) works correctly
- PASS: Learning UI (Learning.tsx, Sidebar.tsx, TopicContent.tsx) renders content correctly
- FAIL BUG-03: All questions hardcoded in aptitudeData.ts — same 5 questions every quiz
- FAIL: No backend API — apps/aptitude/ and apps/learning/ are completely empty
- FAIL: No admin panel to add/edit questions without touching code
- FAIL: No progress tracking saved to server — saveProgress() only writes to localStorage

7.2 Aptitude Backend Models

```
# apps/aptitude/models.py
from django.db import models
from django.conf import settings

class AptitudeTopic(models.Model):
    name = models.CharField(max_length=100, unique=True)
    icon = models.CharField(max_length=10, default='[chart]')
    description = models.TextField(blank=True)
    learning_html = models.TextField(blank=True)  # Rich HTML content
    video_url = models.URLField(blank=True)        # YouTube embed URL
    difficulty = models.CharField(max_length=20, default='Beginner')
    order = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['order', 'name']

class AptitudeQuestion(models.Model):
    topic = models.ForeignKey(AptitudeTopic, on_delete=models.CASCADE,
                              related_name='questions')
    question_text = models.TextField()
    option_a = models.CharField(max_length=300)
    option_b = models.CharField(max_length=300)
    option_c = models.CharField(max_length=300)
    option_d = models.CharField(max_length=300)
    correct_answer = models.CharField(max_length=1,
        choices=[('A','A'),('B','B'),('C','C'),('D','D')])
    explanation = models.TextField(blank=True)
    difficulty = models.CharField(max_length=20, default='Medium')
    is_active = models.BooleanField(default=True)

class QuizAttempt(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    topic = models.ForeignKey(AptitudeTopic, on_delete=models.CASCADE)
    score = models.FloatField()             # 0-100
    total_questions = models.IntegerField()
    correct_answers = models.IntegerField()
    time_taken_seconds = models.IntegerField(null=True, blank=True)
    answers = models.JSONField(default=dict)  # {question_id: 'A/B/C/D'}
    attempted_at = models.DateTimeField(auto_now_add=True)
```

7.3 Random Question API — The Critical Fix

The key change is ORDER BY RANDOM() so users see different questions each attempt:

```
# apps/aptitude/views.py
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def get_questions(request):
    topic_id = request.query_params.get('topic_id')
    count = int(request.query_params.get('count', 10))
    difficulty = request.query_params.get('difficulty', 'all')
    qs = AptitudeQuestion.objects.filter(topic_id=topic_id, is_active=True)
    if difficulty != 'all':
        qs = qs.filter(difficulty=difficulty)
    # THE KEY FIX: ORDER BY RANDOM() — different questions every attempt
    qs = qs.order_by('?').values(
        'id', 'question_text', 'option_a', 'option_b', 'option_c', 'option_d'
        # NOTE: Do NOT include correct_answer — send after submit only
    )[:min(count, 20)]
    return Response(list(qs))
```

8. AI Interview Module — Complete Redesign Blueprint

The interview module has 5 critical bugs (BUG-04 through BUG-09). Rather than patch them individually, the module needs a structured redesign with 4 custom hooks and a state machine.

8.1 Current vs Target Architecture

Aspect	Current (Broken)	Target (Industry Level)
Speech Input	MediaRecorder -> audio blob -> POST /transcribe/ -> Whisper API (latency + server cost)	Web Speech API (browser built-in, free, zero latency, real-time transcript)
Speech Output	speakQuestion() with no Chrome pause-fix, no voice-loaded wait	useTTS hook with keep-alive interval + onvoiceschanged listener
Silence Detection	None — user must manually click Submit	useSilenceDetector: 3s of no new transcript auto-advances
Answer Storage	In-memory React state only — lost on refresh	useInterviewSession: every answer saved to localStorage immediately
Evaluation Timing	Gemini called per answer — 5 calls, 5 waits	Single Gemini call at session end with all 8 Q&A pairs
UI States	Single isLoading boolean — no phase awareness	State machine with 11 named phases
Error Recovery	Page refresh = lost session	localStorage backup allows full session recovery on refresh
PDF Report	No PDF feature	jsPDF + html2canvas captures results div as PDF

8.2 New Interview Session Flow — 20 Steps

1. User navigates to /ai-interview -> AIInterviewLanding.tsx shown
2. User selects interview type + uploads/selects resume -> navigates to /ai-interview-session
3. InterviewSession.tsx mounts -> BrowserCheck phase -> detect window.SpeechRecognition support
4. MicPermission phase -> navigator.mediaDevices.getUserMedia({audio:true}) -> confirm mic access
5. If session exists in localStorage (refresh scenario) -> restore from localStorage
6. POST /api/interview/start -> backend generates all 8 questions at once from Gemini
7. PreBrief phase -> show candidate name, question count, HR vs Technical split, 4 rules
8. User clicks 'Begin' -> runQuestion(0) called
9. SPEAKING phase -> useTTS.speak() reads question aloud -> await Promise.resolve(onend)
10. COUNTDOWN phase -> 2 second visual countdown (2...1...)
11. RECORDING phase -> Web Speech API starts -> live transcript shown -> useSilenceDetector starts
12. 3 seconds of silence -> finalizeAnswer() called -> stop recording -> save to localStorage
13. SAVING_ANSWER phase -> 800ms 'Answer Saved' flash
14. runQuestion(currentIdx + 1) -> repeat steps 9-13 for all 8 questions
15. REVIEW phase -> show all 8 Q&A pairs from localStorage -> user can re-record any answer
16. User clicks 'Submit for AI Evaluation' -> SUBMITTING phase

17. POST /api/interview/submit-all with all 8 Q&A pairs -> Gemini evaluates holistically in ONE call
18. RESULTS phase -> InterviewResults.tsx shows full evaluation with scores, charts, feedback
19. User clicks 'Download PDF' -> generateInterviewPDF() captures div -> saves PDF file
20. localStorage cleared for interview keys

8.3 Custom Hooks

8.3.1 useTTS.ts — Text-to-Speech (fixes BUG-06 & BUG-07)

```
// src/hooks/useTTS.ts
import { useRef, useCallback, useEffect } from 'react';

export function useTTS() {
  const synth = useRef(window.speechSynthesis);
  const getVoice = () => {
    const voices = synth.current.getVoices();
    return voices.find(v => v.name.includes('Google') && v.lang.startsWith('en'))
      || voices.find(v => v.lang.startsWith('en-IN'))
      || voices.find(v => v.lang.startsWith('en'))
      || null;
  };
  const speak = useCallback((text: string): Promise<void> => {
    return new Promise((resolve) => {
      synth.current.cancel();
      const utter = new SpeechSynthesisUtterance(text);
      utter.rate = 0.88; utter.pitch = 1.0; utter.volume = 1.0;
      const v = getVoice(); if (v) utter.voice = v;
      // BUG-06 FIX: Chrome pause-resume ping every 5s for long speech
      const ping = setInterval(() => {
        if (synth.current.paused) synth.current.resume();
      }, 5000);
      utter.onend = () => { clearInterval(ping); resolve(); };
      utter.onerror = () => { clearInterval(ping); resolve(); };
      synth.current.speak(utter);
    });
  }, []);
  // BUG-07 FIX: Wait for voices to load
  useEffect(() => {
    if (synth.current.getVoices().length === 0) {
      synth.current.onvoiceschanged = () => {};
    }
    return () => synth.current.cancel();
  }, []);
  return { speak, stopSpeaking: () => synth.current.cancel() };
}
```

8.3.2 useSilenceDetector.ts — 3-Second Auto-Advance (fixes BUG-08)

```
// src/hooks/useSilenceDetector.ts
import { useRef, useCallback } from 'react';

export function useSilenceDetector(onSilence: () => void, silenceMs = 3000) {
  const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const activeRef = useRef(false);
  const resetSilenceTimer = useCallback(() => {
    if (!activeRef.current) return;
    if (timerRef.current) clearTimeout(timerRef.current);
    timerRef.current = setTimeout(() => {
      if (activeRef.current) onSilence();
    }, silenceMs);
  }, [onSilence, silenceMs]);
  const startSilenceDetection = useCallback(() => {
    activeRef.current = true; resetSilenceTimer();
  }, [resetSilenceTimer]);
  const stopSilenceDetection = useCallback(() => {
    activeRef.current = false;
```

```
    if (timerRef.current) clearTimeout(timerRef.current);
  }, []);
  return { startSilenceDetection, stopSilenceDetection, resetSilenceTimer };
}
```

8.3.3 useInterviewSession.ts — localStorage Persistence (fixes BUG-09)

```
// src/hooks/useInterviewSession.ts
const K_SESSION = 'iv_session';
const K_ANS = (id: string) => `iv_ans_${id}`;
const K_IDX = 'iv_current_index';

export function useInterviewSession() {
  const saveSession = (sessionId: string, questions: any[]) => {
    localStorage.setItem(K_SESSION, JSON.stringify({ sessionId, questions }));
    localStorage.setItem(K_IDX, '0');
  };
  const loadSession = () => {
    try { const r = localStorage.getItem(K_SESSION);
          return r ? JSON.parse(r) : null; }
    catch { return null; }
  };
  const saveAnswer = (qId:string, qText:string, qType:string, answer:string) => {
    localStorage.setItem(K_ANS(qId), JSON.stringify({
      questionId: qId, questionText: qText, questionType: qType,
      answerText: answer, timestamp: new Date().toISOString()
    }));
  };
  const loadAllAnswers = (questions: any[]) => questions.map(q => {
    try { const r = localStorage.getItem(K_ANS(q.id));
          return r ? JSON.parse(r) : { questionId: q.id, answerText: '[No answer]' };
    } catch { return { questionId: q.id, answerText: '[No answer]' }; }
  });
  const clearSession = (questions: any[]) => {
    localStorage.removeItem(K_SESSION);
    localStorage.removeItem(K_IDX);
    questions.forEach(q => localStorage.removeItem(K_ANS(q.id)));
  };
  return { saveSession, loadSession, saveAnswer, loadAllAnswers, clearSession };
}
```

8.4 State Machine — 11 Phases

```
// Phase constants — replaces all isLoading/isSubmitting booleans
const PHASES = {
  BROWSER_CHECK    : 'browser_check',
  MIC_PERMISSION   : 'mic_permission',
  LOADING_QUESTIONS: 'loading_questions',
  PRE_BRIEF        : 'pre_brief',
  SPEAKING         : 'speaking',       // TTS reading question aloud
  COUNTDOWN        : 'countdown',      // 2s visual countdown after TTS
  RECORDING        : 'recording',      // User is speaking
  SAVING_ANSWER    : 'saving_answer',  // 800ms 'Answer Saved' flash
  REVIEW           : 'review',         // Show all 8 answers pre-submit
  SUBMITTING       : 'submitting',     // POST /submit-all in progress
  RESULTS          : 'results',
  ERROR            : 'error',
} as const;
```

8.5 Backend — New /submit-all Endpoint

```
# Add to apps/interview/views.py
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def submit_all_answers(request):
    session_id = request.data.get('session_id')
```

```
    answers = request.data.get('answers', [])
    if not answers or len(answers) < 3:
        return Response({'error': 'Minimum 3 answers required'}, status=400)
    session = get_object_or_404(InterviewSession, id=session_id, user=request.user)
    if session.status == 'completed':
        return Response({'error': 'Session already evaluated'}, status=409)
    try:
        gemini = GeminiService()
        evaluation = gemini.evaluate_full_interview(answers)
    except Exception as e:
        return Response({'error': f'Evaluation failed: {str(e)}'}, status=503)
    session.status = 'completed'
    session.end_time = timezone.now()
    session.overall_score = evaluation.get('overall_score', 0) * 10
    scores = evaluation.get('scores', {})
    session.communication_score = scores.get('communication', 0) * 10
    session.technical_score = scores.get('technical', 0) * 10
    session.confidence_score = scores.get('confidence', 0) * 10
    session.save()
    return Response(evaluation, status=200)
```

9. Module Blueprint — Dashboard

The dashboard currently displays placeholder data. It needs to fetch real stats from the backend APIs.

9.1 Dashboard Data Sources

Widget	API Endpoint	Data Needed
Placement Readiness Score	GET /api/interview/stats/	Latest session's placement_readiness field
Total Interviews	GET /api/interview/stats/	total_interviews count
Average Score	GET /api/interview/stats/	average_score (0-100)
Best Score	GET /api/interview/stats/	best_score (0-100)
Score Trend Chart	GET /api/interview/history/	Array of session overall_scores over time
By Interview Type	GET /api/interview/stats/	by_type: {HR:{count,avg}, Technical:{count,avg}}
Aptitude Topic Progress	GET /aptitude/history/	Per-topic quiz scores
Recent Sessions	GET /api/interview/history/?page_size=3	Latest 3 sessions with type, score, date

9.2 Dashboard Component Changes

- Replace all hardcoded numbers in DashboardHome.tsx with useEffect + axios calls to the endpoints above
- Use Recharts (already installed in package.json) for the score trend line chart
- Add loading skeletons while data loads — do not show zeros, show spinner
- Cache API results in Zustand store for 5 minutes to avoid refetching on every tab switch

10. Environment Configuration

10.1 Backend — backend/.env

```
# backend/.env — Create this file, add to .gitignore

# Django
SECRET_KEY=your-long-random-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# Database (leave blank for SQLite in dev)
DATABASE_URL=sqlite:///db.sqlite3

# Google Gemini AI — Get free key from aistudio.google.com
GEMINI_API_KEY=AIzaSy...

# JWT
JWT_ACCESS_TOKEN_LIFETIME=60     # minutes
JWT_REFRESH_TOKEN_LIFETIME=7     # days

# File uploads
MAX_UPLOAD_SIZE=5242880          # 5MB in bytes
MEDIA_ROOT=./media/

# CORS — allow React dev server
CORS_ORIGINS=http://localhost:5173,http://localhost:3000
```

10.2 Frontend — frontend/.env

```
# frontend/.env
VITE_API_BASE_URL=http://localhost:8000/api

# BUG-11 FIX: All API files must use this instead of hardcoded localhost
# In interviewAPI.ts: const BASE = import.meta.env.VITE_API_BASE_URL + '/interview'
# In authService.ts:  const BASE = import.meta.env.VITE_API_BASE_URL
```

10.3 settings.py — Required Updates

```
# Add to ai_trainer/settings.py
from decouple import config

GEMINI_API_KEY = config('GEMINI_API_KEY', default='')
DATA_UPLOAD_MAX_MEMORY_SIZE = 5 * 1024 * 1024  # 5MB
FILE_UPLOAD_MAX_MEMORY_SIZE = 5 * 1024 * 1024
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
CORS_ALLOWED_ORIGINS = config('CORS_ORIGINS',
    default='http://localhost:5173').split(',')
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=config(
        'JWT_ACCESS_TOKEN_LIFETIME', default=60, cast=int)),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=config(
        'JWT_REFRESH_TOKEN_LIFETIME', default=7, cast=int)),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}
```

11. Implementation Roadmap — Week by Week

This roadmap builds the project in dependency order. Each week only begins after the previous week is tested and passing.

Week	Phase	Tasks	Deliverable / Test
Week 1	**Setup & Auth Fixes**	1. Add .env files for backend and frontend 2. Fix BUG-11: hardcoded URLs -> VITE_API_BASE_URL 3. Fix BUG-01: name validation in Register.tsx 4. Fix BUG-02: name validation in serializers.py 5. Update google-generativeai to >=0.7.0	Register with valid/invalid names — invalid chars blocked. API URLs load from env.
Week 2	**Aptitude Backend**	1. Build AptitudeTopic, AptitudeQuestion, QuizAttempt models 2. Run seed_aptitude.py to import all CSV questions 3. Build /aptitude/questions/ endpoint with ORDER BY RANDOM() 4. Update Quiz.tsx to fetch from API 5. Add URL to ai_trainer/urls.py	Quiz shows different questions every attempt. At least 40+ questions imported.
Week 3	**Custom Hooks**	1. Create src/hooks/ folder 2. Build useTTS.ts (BUG-06 + BUG-07 fixes) 3. Build useSTT.ts (BUG-05 fix) 4. Build useSilenceDetector.ts (BUG-08 fix) 5. Build useInterviewSession.ts (BUG-09 fix) 6. Test each hook in isolation	All 4 hooks work independently. TTS reads text. STT transcribes live. Silence auto-triggers after 3s.
Week 4	**Interview Backend**	1. Add generate_all_questions() to GeminiService 2. Add evaluate_full_interview() with holistic prompt 3. Modify start_interview view to generate all 8 questions at once 4. Add submit_all_answers view (BUG-04 fix) 5. Register new URL 6. Test with Postman	POST /start returns 8 questions. POST /submit-all accepts 8 Q&A, returns holistic JSON.
Week 5	**New Interview UI**	1. Create all missing interview components 2. Rewrite InterviewSession.tsx as state machine with 11 phases 3. Wire all 4 hooks into state machine 4. Add session recovery on refresh 5. Build AnswerReview.tsx pre-submit screen	Complete voice interview flow in Chrome: TTS -> countdown -> auto-record -> silence -> next Q -> review -> submit.
Week 6	**Results + PDF**	1. Build InterviewResults.tsx with all score components 2. Build ReportTemplate.tsx with inline styles 3. Implement pdfGenerator.ts using jsPDF + html2canvas 4. Install jspdf and html2canvas 5. Add Download button to results page	Results dashboard shows all scores. PDF downloads correctly with all 8 Q+A+feedback.
Week 7	**Dashboard & Polish**	1. Connect DashboardHome.tsx to real API data 2. Add Recharts score trend chart 3. Fix BUG-12: unify token storage 4. Final testing: all 16 bugs verified fixed 5. Test full flow: register -> upload resume -> interview -> PDF	Full end-to-end flow works. All bugs in inventory show resolved.

12. Bug Fix Status Tracker

Use this table as a daily checklist. Check off each item only after the fix is implemented AND tested — not just when the code looks correct.

Bug ID	File	Fix Location	How to Test
BUG-01	Register.tsx	Add onChange regex in Register.tsx	Type '123abc' in name field — chars should be blocked
BUG-02	serializers.py	Add validate() name regex check	POST /auth/register/ with name='123@$' — get 400
BUG-03	aptitudeData.ts	Move to DB + add ORDER BY RANDOM()	Take same quiz twice — different questions shown
BUG-04	InterviewSession.tsx	Add /submit-all endpoint + use in frontend	Check Network tab: only 1 API call after submit
BUG-05	InterviewSession.tsx	Replace MediaRecorder with Web Speech API	Speak a sentence — see live transcript appear
BUG-06	InterviewSession.tsx	Add setInterval ping in useTTS.ts	Read a 200-word text aloud — it should not pause
BUG-07	InterviewSession.tsx	Add onvoiceschanged in useTTS.ts	TTS on first load should sound like Google voice
BUG-08	InterviewSession.tsx	useSilenceDetector 3s timeout	Stop speaking for 3s — question auto-advances
BUG-09	InterviewSession.tsx	useInterviewSession localStorage saves	Refresh page mid-interview — session restores
BUG-10	views.py	Decouple answer save and question generation	Submit answer — even if next Q generation fails, answer is saved
BUG-11	interviewAPI.ts / authService.ts	Replace hardcoded URLs with env var	Change VITE_API_BASE_URL — app should point to new URL
BUG-12	authStore.ts	Store both tokens consistently	Refresh page after login — user should still be logged in
BUG-13	requirements.txt	Upgrade google-generativeai to >=0.7.0	Run pip install and verify no import errors
BUG-14	apps/aptitude/	Implement full aptitude backend module	GET /aptitude/questions/ returns questions
BUG-15	apps/learning/	Implement learning backend module	GET /learning/topics/ returns topics
BUG-16	InterviewFeedback.tsx	Standardize 0-10 scale, display x10	Results show 7.5/10 not 75/100 inconsistency

Appendix A — Quick Start Commands

Run these commands in order after cloning the project to get the development environment running.

```
# BACKEND SETUP
cd ai-trainer/backend
python -m venv venv
source venv/bin/activate          # Linux/Mac
venv\Scripts\activate            # Windows

pip install -r requirements.txt
pip install 'google-generativeai>=0.7.0'

# Create .env file with your GEMINI API KEY (see Section 10.1)

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py shell < ../../data set/seed aptitude.py
python manage.py runserver 8000

# FRONTEND SETUP
cd ai-trainer/frontend
npm install
npm install jspdf html2canvas
echo 'VITE_API_BASE_URL=http://localhost:8000/api' > .env
npm run dev
# App is now running at http://localhost:5173
```

Appendix B — Key File Reference

Task	File to Open	Section
Fix name validation	frontend/src/components/auth/Register.tsx	Section 6.2
Fix serializer validation	backend/apps/accounts/serializers.py	Section 6.3
Build aptitude models	backend/apps/aptitude/models.py	Section 7.2
Build random questions API	backend/apps/aptitude/views.py	Section 7.3
Seed questions from CSV	data_set/seed_aptitude.py (create)	Section 7.4
TTS hook	frontend/src/hooks/useTTS.ts (create)	Section 8.3.1
STT hook	frontend/src/hooks/useSTT.ts (create)	Section 8.3.2
Silence detection hook	frontend/src/hooks/useSilenceDetector.ts (create)	Section 8.3.3
Session localStorage hook	frontend/src/hooks/useInterviewSession.ts (create)	Section 8.3.4
Interview state machine	frontend/src/pages/InterviewSession.tsx (rewrite)	Section 8.4

Backend submit-all endpoint	backend/apps/interview/views.py (add)	Section 8.5
Gemini holistic prompt	backend/apps/interview/services/gemini_service.py	Section 8.6
PDF generator	frontend/src/utils/pdfGenerator.ts (create)	Section 11 Week 6
Environment variables	backend/.env + frontend/.env (create)	Section 10
Dashboard real data	frontend/src/components/dashboard/sections/DashboardHome.tsx	Section 9

End of Blueprint Document

AI Pre-Placement Trainer | Blueprint v1.0 | Based on complete code review of AI-BASED-TRAINER-FEEDBACK-MODEL-main.zip

YSPM's Yashoda Technical Campus, Satara | AI & Data Science Department | Academic Year 2025-26